UNIVERSIDAD DE MARGARITA
Alma Mater del Caribe.
Vicerrectorado Académico.
Decanato de Ingeniería y Afines.
Unidad Curricular: Programación I.

INFORME TÉCNICO.

Kit Multifuncional de Automatización de Archivos.

Profesor:

Richard Rivera.

Realizado por:

Gabriel Rodríguez.

Sebastian Rodriguez.

Sebastian Matto.

El Valle del Espíritu Santo, 23 de julio del 2026.

1. Descripción general del sistema

El Kit Multifuncional de Automatización de Archivos es un sistema desarrollado en Python 3, ejecutable exclusivamente desde la línea de comandos, que permite organizar, analizar, auditar y generar reportes sobre una carpeta del sistema de archivos. El sistema aplica un enfoque de programación estructurada y modular: cada responsabilidad del sistema (organizar, analizar, auditar y reportar) vive en su propio archivo Python, y todos los módulos se integran a través de un menú principal interactivo implementado en main.py.

El sistema resuelve un problema realista: dada una carpeta con archivos heterogéneos (documentos, imágenes, hojas de cálculo, logs, etc.), permite clasificarlos automáticamente, buscar información relevante dentro de su contenido mediante expresiones regulares, detectar cambios entre distintos momentos en el tiempo (auditoría por snapshots) y dejar constancia de todo en reportes técnicos legibles en formato .txt y .csv.

Los objetivos específicos cubiertos por la implementación son:

- Menú interactivo en línea de comandos con submenús por módulo y validación de entrada.
- Clasificación de archivos por extensión, tamaño y fecha de modificación.
- Renombrado de archivos mediante patrones definidos por el usuario con expresiones regulares.
- Búsqueda de correos, teléfonos y fechas dentro de archivos de texto usando regex.
- Sistema de auditoría basado en snapshots (hash MD5) que detecta archivos nuevos, modificados y eliminados.
- Generación de reportes automáticos en formato .txt y .csv.
- Uso de decoradores, generadores, manejo de excepciones y estructuras de datos (listas, diccionarios, tuplas y conjuntos).

Además de los requisitos obligatorios, el equipo agregó un conjunto de mejoras para hacer el sistema más robusto y agradable de usar en la práctica:

- Banner ASCII a color al iniciar y mensajes de consola con código de colores según el tipo (éxito, error, aviso).
- Barra de progreso animada mientras se clasifican o renombran archivos.
- Señales sonoras de éxito/error/aviso (con alternativa segura en sistemas sin soporte de sonido).
- Gráficos de barras (.png) opcionales con matplotlib para visualizar resultados de organización, análisis o auditoría.
- Sistema de Deshacer (undo.py): revierte la última operación de organización (mover o renombrar) por si el usuario se equivoca.
- Verificación de permisos de escritura al iniciar, con mensajes claros para el problema más común en Windows (carpetas protegidas contra ransomware).

2. Arquitectura y propósito de cada módulo

El sistema está organizado en 6 archivos Python independientes. La siguiente tabla resume el propósito de cada uno:

Archivo / Carpeta	Tipo	Descripción
main.py	Módulo principal	Contiene el menú principal y los submenús de cada módulo. Muestra el banner ASCII, verifica permisos de escritura al iniciar, y coordina sonido y reportes.
organizer.py	Módulo a)	Gestor de Organización de Archivos: clasifica por extensión, tamaño y fecha; renombra con regex; soporta dry-run; registra cada cambio real en undo.py.
analyzer.py	Módulo b)	Analizador de Contenido: busca patrones (correos, teléfonos, fechas) dentro de archivos de texto usando regex y genera resúmenes de frecuencia.
auditor.py	Módulo c)	Auditor de Cambios: genera snapshots del estado de una carpeta (hash MD5 por archivo) y los compara para detectar nuevos, modificados y eliminados.
reports.py	Módulo d)	Generador de Reportes: crea reportes .txt y .csv, y opcionalmente un gráfico de barras .png (matplotlib) a partir de los resultados de los demás módulos.
undo.py	Módulo e) (extra)	Sistema de Deshacer: registra cada movimiento/renombrado real en historial_operaciones.json y permite revertir la última operación de organización.
utils.py	Módulo de soporte	Validaciones, el decorador registrar_actividad, los generadores leer_lineas()/listar_archivos(), y las utilidades de interfaz: banner ASCII, sonido, barra de progreso y verificación de permisos.

2.1 Diagrama de flujo general del sistema

El siguiente diagrama muestra el ciclo de vida general del programa: desde que se muestra el menú principal hasta que el usuario elige un módulo, se ejecuta la operación correspondiente y el sistema regresa al menú (o finaliza).

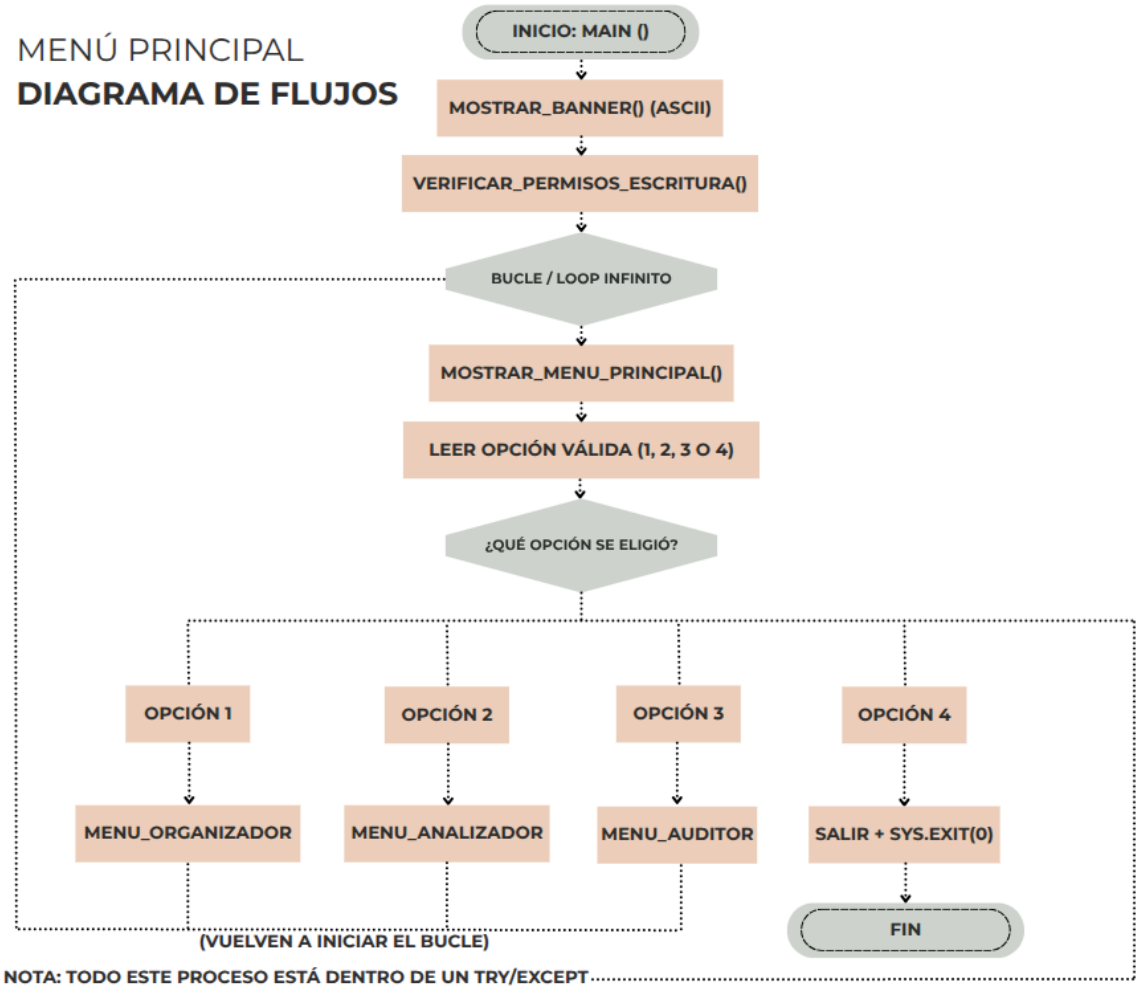


Figura 1. Diagrama de flujo general (main.py)

2.2 Diagrama de flujo - Gestor de Organización de Archivos

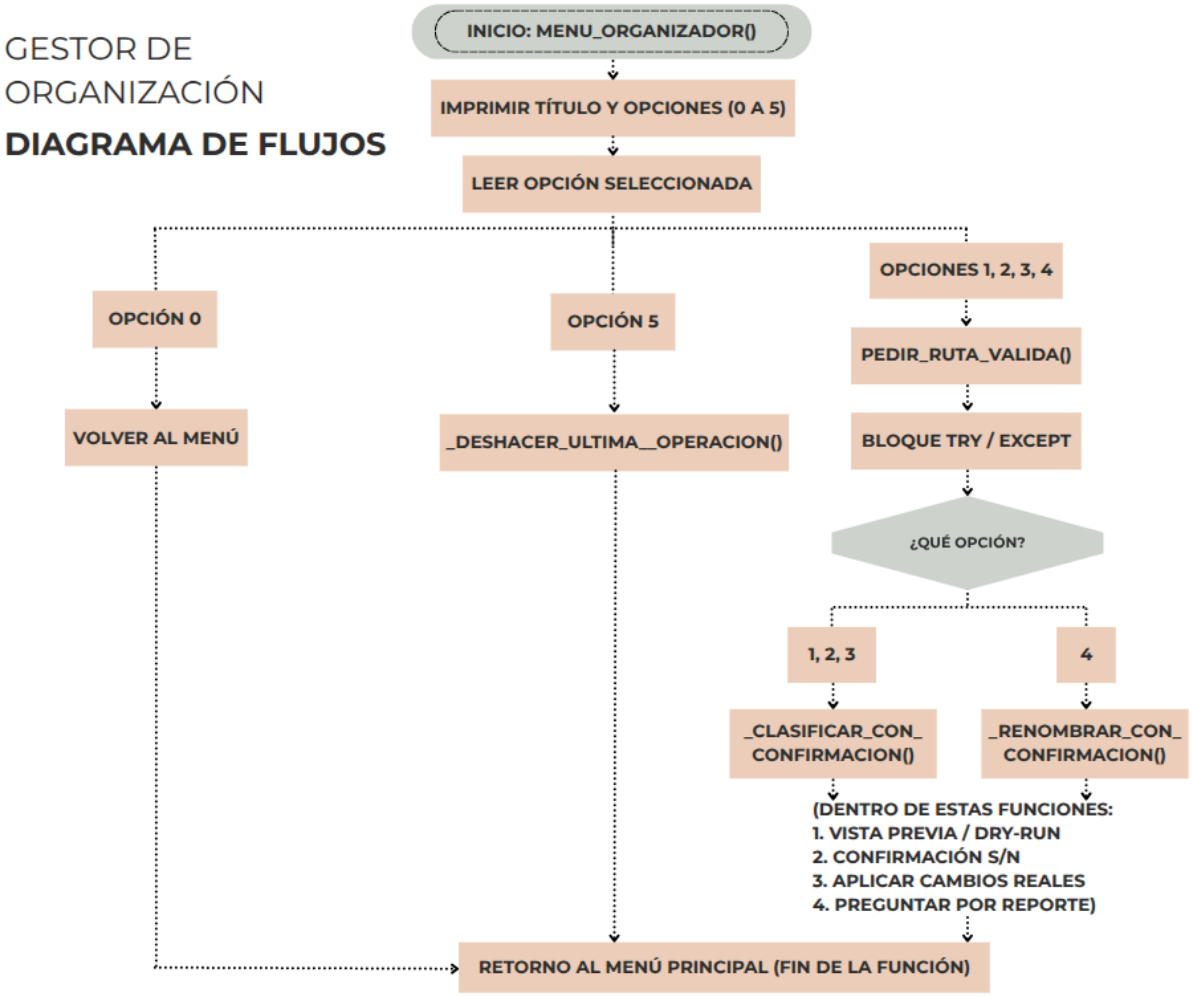


Figura 2. Diagrama de flujo del módulo organizer.py (incluye la opción de deshacer)

2.3 Diagrama de flujo - Analizador de Contenido

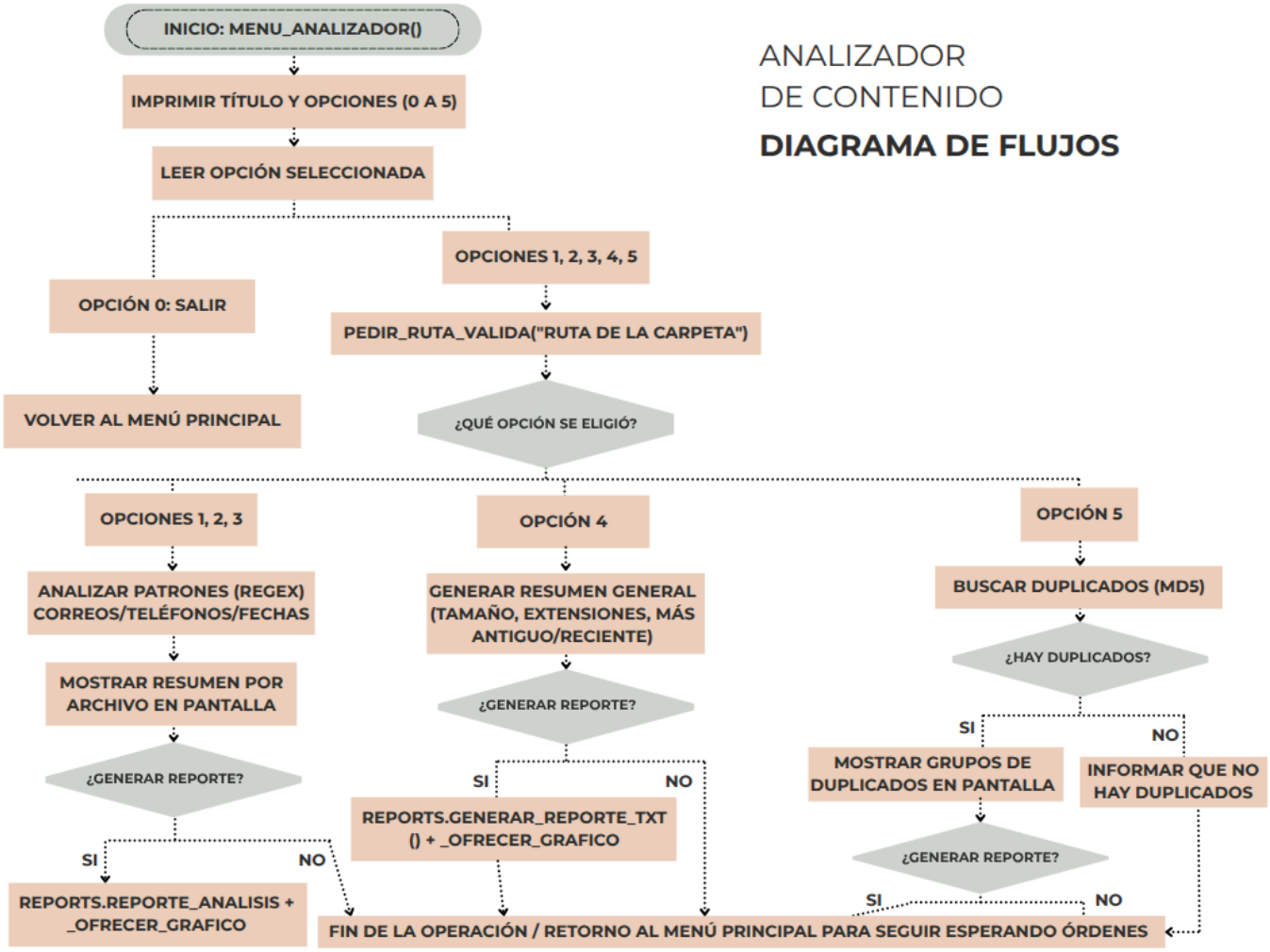


Figura 3. Diagrama de flujo del módulo analyzer.py

2.4 Diagrama de flujo - Auditor de Cambios

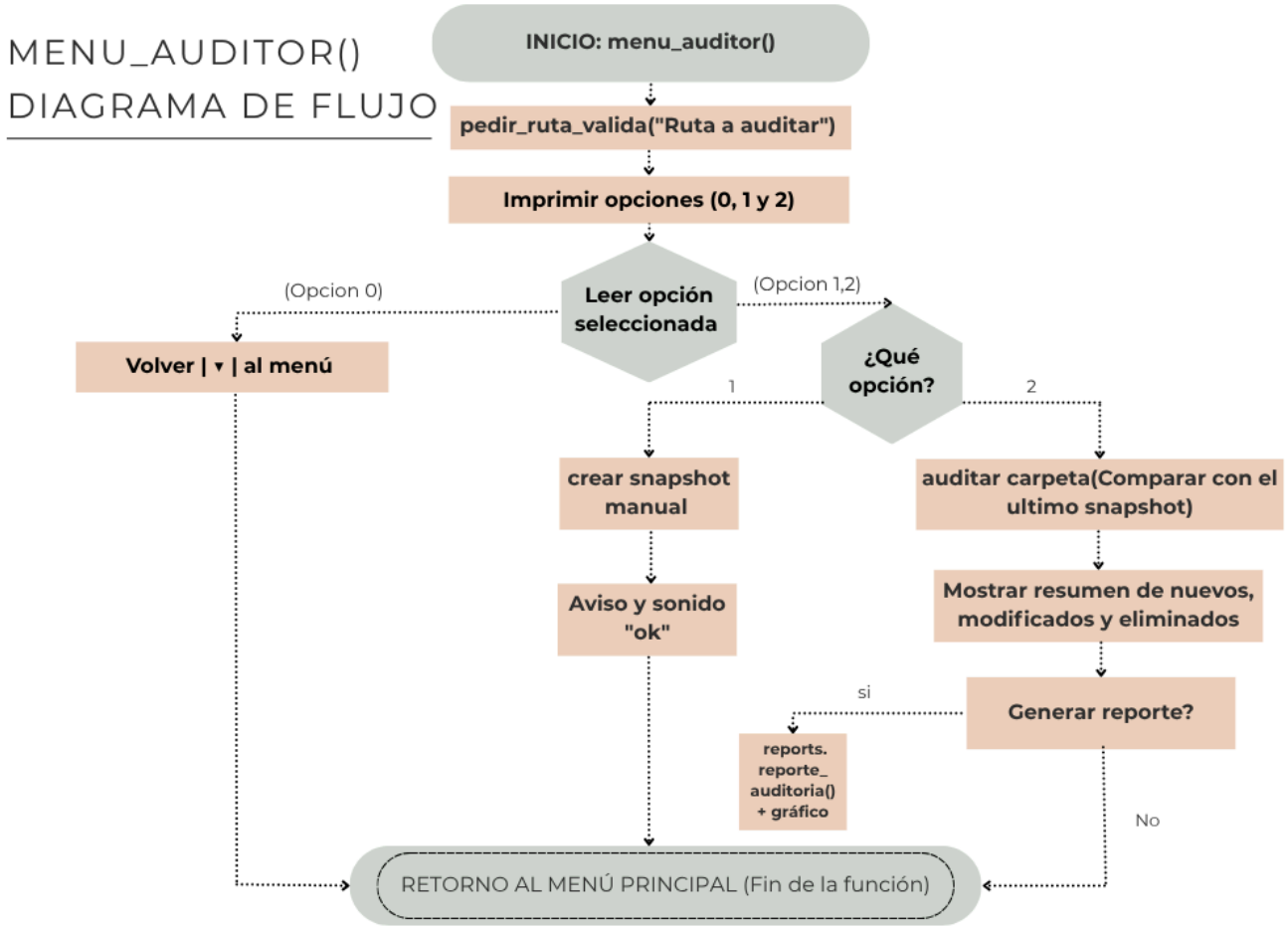


Figura 4. Diagrama de flujo del módulo auditor.py

2.5 Diagrama de flujo - Generador de Reportes

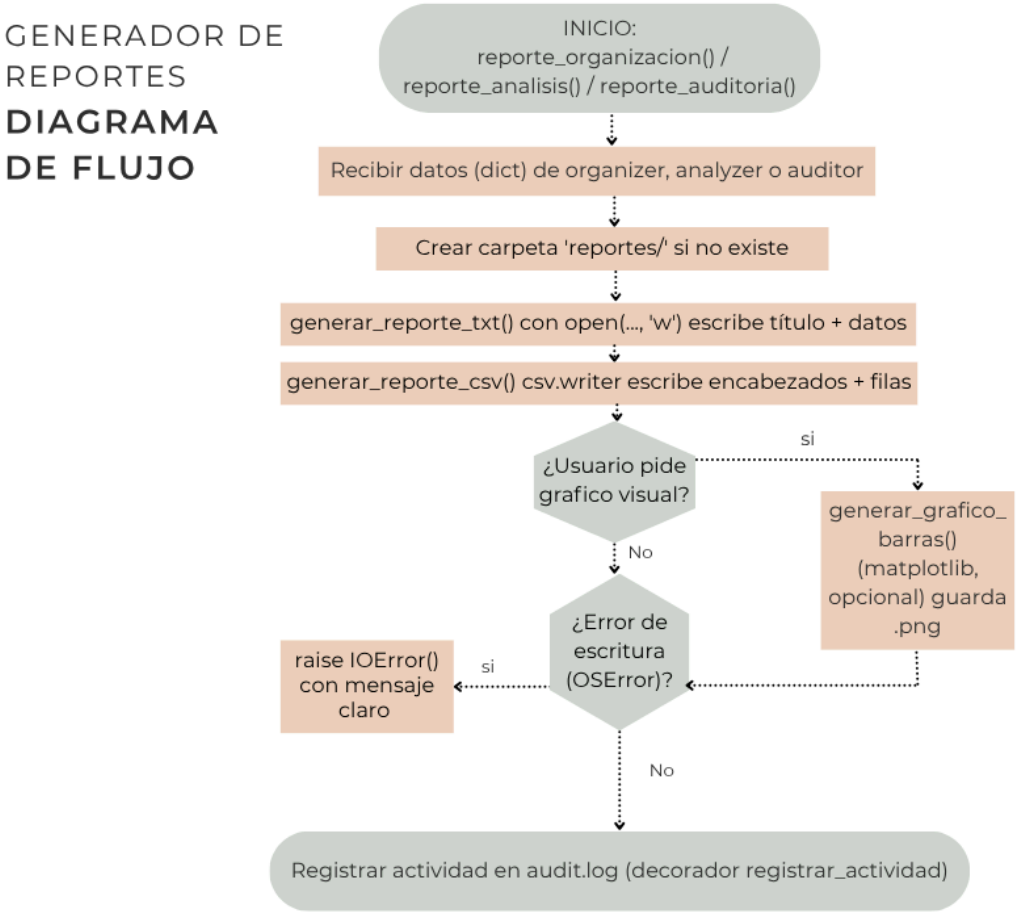


Figura 5. Diagrama de flujo del módulo reports.py (incluye gráfico opcional)

2.6 Diagrama de flujo - Sistema de Deshacer (undo.py)

Este módulo es una funcionalidad extra agregada por el equipo: revierte la última operación de organización (mover o renombrar archivos) leyendo el historial guardado en historial_operaciones.json.

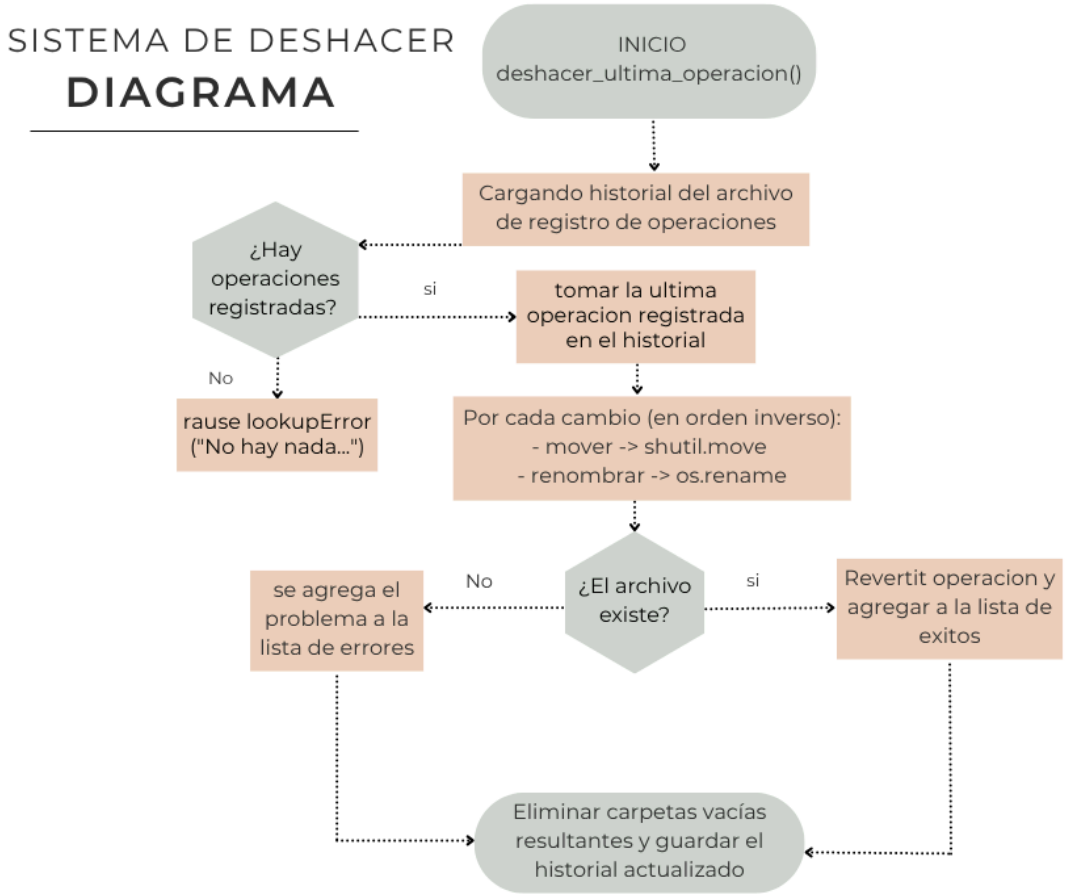


Figura 6. Diagrama de flujo del módulo undo.py

2.7 Diagrama de casos de uso general

El diagrama de casos de uso resume la interacción del actor 'Usuario' con las funcionalidades principales del sistema. Las relaciones <<extend>> indican que el modo de simulación (dry-run) y el sistema de deshacer son variantes opcionales del caso de uso 'Organizar archivos'. Las relaciones <<include>> indican que los tres módulos principales pueden, opcionalmente, invocar la generación de reportes.

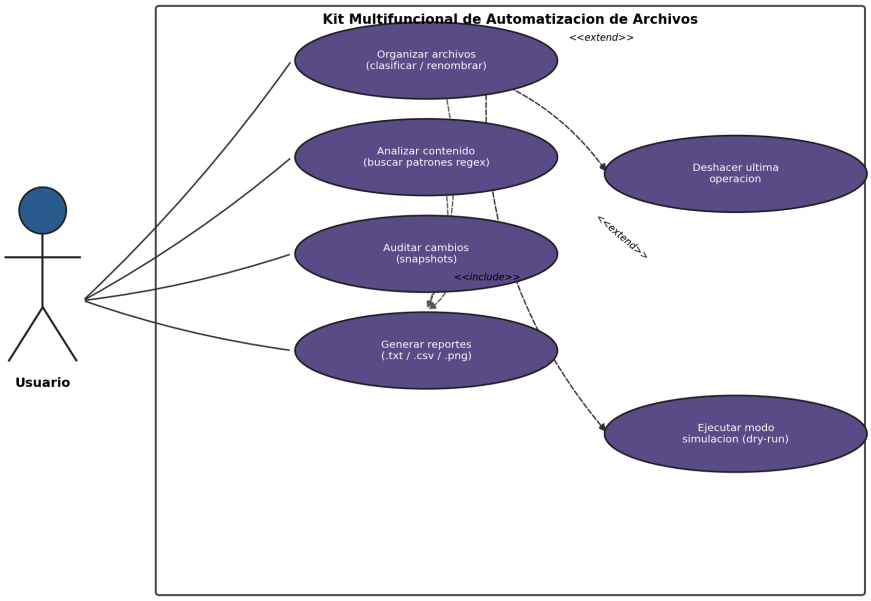


Figura 7. Diagrama de casos de uso general del sistema

3. Descripción y firma de cada función implementada

3.1 utils.py

Función	Parámetros	Retorna	Descripción
validar_carpeta(ruta)	ruta: str	bool	Valida que la ruta exista y sea un directorio; lanza FileNotFoundError o NotADirectoryError si no es válida.
pedir_opcion(mensaje, opciones_validas)	mensaje: str, opciones_validas: set	str	Solicita al usuario una opción dentro de un conjunto válido, repitiendo hasta recibir una entrada correcta.
pedir_ruta_valida(mensaje)	mensaje: str	str	Solicita al usuario una ruta de carpeta válida, repitiendo si la ruta no existe.
registrar_actividad(func)	func: function	function (wrapper)	Decorador que registra en audit.log el nombre de la función, la duración de ejecución y si finalizó con éxito o con error.
escribir_log(linea)	linea: str	None	Escribe (append) una línea de texto en audit.log. Si no hay permisos, avisa una sola vez con instrucciones claras.
leer_lineas(ruta_archivo)	ruta_archivo: str	generator de tuplas (int, str)	Generador que lee un archivo línea por línea sin cargarlo completo en memoria.
listar_archivos(carpeta)	carpeta: str	generator de str	Generador que recorre una carpeta (nivel superior) y entrega la ruta de cada archivo.
obtener_metadata(ruta_archivo)	ruta_archivo: str	dict	Devuelve nombre, extensión, tamaño y fecha de modificación de un archivo.
mostrar_banner()	ninguno	None	Imprime el banner ASCII de bienvenida a color, una sola vez al iniciar el programa.
imprimir_titulo(texto)	texto: str	None	Imprime un título de sección dentro de un recuadro de color, usado en los submenús.
reproducir_sonido(tipo)	tipo: str ('ok'/'error'/'aviso')	None	Reproduce un beep corto según el resultado de la operación; usa winsound en Windows y el timbre ASCII en otros sistemas.
con_barra_progreso(items, descripción)	items: list, descripcion: str	generator	Envuelve una lista y muestra una barra de progreso animada en la terminal mientras se itera.
verificar_permisos_escritura()	ninguno	bool	Prueba crear un archivo temporal en la carpeta actual para detectar problemas de permisos antes de operar.

3.2 organizer.py

Las cuatro funciones siguen el mismo patrón: usan con_barra_progreso() de utils.py para mostrar avance, y cuando dry_run es False, registran cada cambio real en undo.py para poder revertirlo después.

Función	Parámetros	Retorna	Descripción
clasificar_por_extension(carpeta, dry_run=False)	carpeta: str, dry_run: bool	dict {ext: [archivos]}	Agrupa archivos por extensión y los mueve a subcarpetas nombradas según la extensión.
clasificar_por_tamano(carpeta, dry_run=False)	carpeta: str, dry_run: bool	dict {categoria: [archivos]}	Clasifica archivos en pequeños, medianos y grandes según su tamaño en bytes.
clasificar_por_fecha(carpeta, dry_run=False)	carpeta: str, dry_run: bool	dict {anio-mes: [archivos]}	Agrupa archivos por su fecha de modificación (formato año-mes).
renombrar_con_patron(carpeta, patrón, reemplazo, dry_run=False)	carpeta, patrón (regex), reemplazo: str, dry_run: bool	list de tuplas (viejo, nuevo)	Renombra archivos cuyo nombre coincide con una expresión regular, sustituyéndola por el texto de reemplazo.

3.3 analyzer.py

Función	Parámetros	Retorna	Descripción
buscar_patron_en_archivo(ruta_archivo, patrón)	ruta_archivo: str, patron: regex	list de tuplas (línea, coincidencia)	Busca un patrón regex dentro de un archivo usando el generador leer_lineas.
extraer_lineas_relevantes(ruta_archivo, palabra_clave)	ruta_archivo: str, palabra_clave: str	list de tuplas (línea, texto)	Extrae las líneas completas que contienen una palabra clave (búsqueda insensible a mayúsculas).
analizar_carpeta(carpeta, tipo_patron='correos')	carpeta: str, tipo_patron: str	dict con resumen, únicos y frecuencias	Analiza todos los archivos de texto de una carpeta buscando correos, teléfonos o fechas.

3.4 auditor.py

Función	Parámetros	Retorna	Descripción
crear_snapshot(carpeta, nombre_snapshot=None)	carpeta: str, nombre_snapshot : str opcional	str (ruta del snapshot)	Crea una fotografía JSON del estado de la carpeta: nombre, tamaño, fecha y hash MD5 de cada archivo.
comparar_snapshots(snapshot_anterior, snapshot_actual)	dos diccionarios	dict con nuevos/modificados/eliminados	Compara dos snapshots y detecta archivos nuevos, modificados (distinto hash) y eliminados.
auditar_carpeta(carpeta)	carpeta: str	dict con diferencias	Flujo completo: carga el último snapshot, crea uno nuevo, compara ambos y registra el resultado en audit.log.

3.5 reports.py

Función	Parámetros	Retorna	Descripción
generar_reporte_txt(datos, nombre_base, título)	datos: dict/list, nombre_base: str, título: str	str (ruta .txt)	Genera un reporte de texto legible con los datos recibidos.
generar_reporte_csv(filas, encabezados, nombre_base)	filas: list, encabezados: list, nombre_base: str	str (ruta .csv)	Genera un reporte CSV con encabezados y filas de datos.
reporte_organizacion / reporte_analisis / reporte_auditoria	resultado del módulo correspondiente	tuple (ruta_txt, ruta_csv)	Funciones de conveniencia que generan ambos formatos de reporte a partir del resultado de cada módulo.
generar_grafico_barras(datos, nombre_base, título, ...)	datos: dict, nombre_base: str, título: str	str (ruta .png) o None	Extra: genera un gráfico de barras con matplotlib. Si matplotlib no está instalado, avisa y retorna None sin interrumpir el programa.

3.6 undo.py (extra)

Función	Parámetros	Retorna	Descripción
registrar_operacion(nombre_operacion, cambios)	nombre_operacion: str, cambios: list de dict	None	Guarda en historial_operaciones.json una operación reversible (movimientos y/o renombrados).
deshacer_ultima_operacion()	ninguno	dict con éxitos/errores	Revierte la última operación registrada: mueve archivos de vuelta a su carpeta original, renombra de vuelta, y elimina carpetas vacías resultantes.
ver_historial(limite=10)	limite: int	list de dict	Retorna las últimas operaciones registradas (la más reciente primero), útil para mostrarle al usuario que se puede deshacer.

4. Archivos y carpetas utilizados y generados

Archivo / Carpeta	Tipo	Descripción
tests_sample/	Entrada	Carpeta de prueba con 7 archivos variados (.txt, .csv, .py, .log, .pdf, .jpg) usada para probar el sistema sin arriesgar archivos reales.
snapshots/*.json	Generado (auditor)	Fotografías JSON del estado de una carpeta en un momento dado; se usan para detectar cambios.
reportes/*.txt , *.csv	Generado (reports)	Reportes legibles con los resultados de clasificación, análisis o auditoría.
reportes/*.png	Generado (reports, extra)	Gráficos de barras opcionales generados con matplotlib para visualizar resultados.
audit.log	Generado (utils)	Bitácora histórica de ejecución: registra cada función crítica invocada, su duración y si tuvo éxito o error.
historial_operaciones.json	Generado (undo, extra)	Historial de operaciones reversibles de organizer.py; permite deshacer la última operación (mover/renombrar).
requirements.txt	Configuracion	Declara matplotlib como dependencia opcional (solo para el gráfico de barras); el núcleo del sistema no requiere librerías externas.

5. Retos encontrados y soluciones aplicadas

Evitar pérdida de archivos al organizar

Se implementó un modo de simulación (dry-run) en todas las funciones de organizer.py, de forma que el usuario pueda revisar el resultado de una clasificación antes de mover archivos de verdad.

Detectar cambios reales y no solo por fecha

Comparar solo la fecha de modificación no es confiable (puede cambiar sin alterar el contenido). Se optó por calcular un hash MD5 del contenido de cada archivo y comparar hashes entre snapshots.

Procesar archivos grandes sin agotar memoria

Se usó un generador (leer_lineas) que entrega el archivo línea por línea con yield, en lugar de cargarlo completo con read() o readlines().

Centralizar el registro de auditoría sin repetir código

Se creó un único decorador (registrar_actividad) en utils.py que se reutiliza en los cuatro módulos, evitando duplicar la lógica de logging y medición de tiempo en cada función.

Validar patrones de expresiones regulares inválidos

Las funciones que reciben un patrón regex del usuario (renombrar_con_patron, buscar_patron_en_archivo) capturan re.error y lo relanzan como ValueError con un mensaje entendible.

Permitir deshacer una operación sin duplicar lógica

En vez de escribir código de reversión en cada función de organizer.py, se centralizó en undo.py: cada función solo reporta qué cambio hizo (mover/renombrar) y undo.py sabe cómo revertir cualquiera de los dos tipos.

Errores de permisos en Windows (carpetas protegidas)

Al probar en Windows, escribir audit.log o crear la carpeta reportes/ fallaba con PermissionError cuando el proyecto estaba dentro de Descargas/Documentos, por la protección contra ransomware de Windows. Se agregó verificar_permisos_escritura() al iniciar el programa, y los mensajes de error ahora explican la causa más probable y cómo resolverla, en vez de repetir el mismo error por cada archivo procesado.

Dependencia opcional (matplotlib) sin romper el núcleo

El gráfico de barras es una funcionalidad extra; se importa matplotlib dentro de un try/except ImportError para que el sistema siga funcionando (sin el gráfico) si la librería no está instalada.

6. Conclusiones del equipo

El desarrollo de este proyecto permitió consolidar, en un caso de uso realista, los conceptos vistos durante el trimestre: funciones y modularidad, estructuras de control, estructuras de datos, manejo de archivos con with open(), manejo de excepciones, expresiones regulares, generadores y decoradores. Dividir el sistema en módulos independientes (organizer, analyzer, auditor, reports y utils) facilitó tanto el desarrollo en equipo como las pruebas de cada funcionalidad por separado.

El uso de un decorador común para el registro de actividad (registrar_actividad) demostró el valor de evitar código repetido, mientras que el sistema de auditoría basado en hashes MD5 permitió detectar cambios de forma más confiable que comparar únicamente fechas de modificación. En conjunto, el proyecto deja una base sólida y reutilizable de automatización de archivos desde la línea de comandos.